MAY 23, 2025

LIVERPOOL HOPE UNIVERSITY

1844

INDUSTRIAL ROBOTICS COURSEWORK

AUTONOMOUS FIRE-FIGHTING ROBOT

SAFEER AHMED

MSC ROBOTICS ENGINEEERING

Abstract

Fire incidents can cause serious threats, resulting in loss of life, property damage, and permanent disabilities. Firefighters risk their life in managing such emergencies. With advances in science and technology, deploying robots in hazardous environments keep firefighters out of harm ways. This report presents the design, simulation and testing of an autonomous firefighting robot that can detect and extinguish small fires in indoor environment. The designing and simulation entirely done in Tinkercad online simulator to validate its functionality. This system is built around Arduino Uno microcontroller and is equipped with flame sensors to detect fires in real time, two ultrasonic sensors to avoid obstacles as it drives itself to the source, and a water spray mechanism attached with two servo motors, one for nozzle direction for fire suppression and other for steering control. Simulation results demonstrate successfully detection of fire and navigation autonomously, highlighting the robot's potential for use in fire-prone areas to enhance safety and reduce human risk and property loss.

Additionally, the robot can be fitted with a camera, GPS and live monitoring system that enables operators to track its status and, if needed, can take control remotely to drive it down exactly to the location or source of fire. These additional capabilities can be crucial when dealing with fires in tight or inaccessible locations. By adopting this approach, industries can better manage fire emergencies, enhancing workplace safety and operational effectiveness. Overall, this work highlights how robotics can revolutionize fire protection and emergency response, setting the stage for future innovations in autonomous firefighting.

Acronym

GPS – Global Positioning System

LDR – Light Dependent Resistor

LED – Light Emitting Diode

IC – Integrated Circuit

MCU – Microcontroller Unit

PWM – Pulse Width Modulation

Table of contents

Abstract 1
1. Chapter: Introduction and background 4
1.1. Importance of autonomous fire-fighting 4
1.2. Context and literature 4
1.3. Project objective 5
2. Material and methods 6
2.1. Hardware components 6
2.2. Circuit design 7
2.3. Control logic and software 9
3. Results & Discussion 13
3.1. Simulation outcomes 13
3.2. System strengths 14
3.3. Limitations and future work 14
3.4. Applications 15
4. Conclusion 16
References 17

Autonomous Fire-Fighting Robot Using Tinkercad

1. Chapter: Introduction and background

1.1. Importance of autonomous fire-fighting

Fire emergencies often expose responders to extreme heat, toxic gases, and collapsing structures, placing human lives at serious risk. Autonomous fire-fighting robots present a safer alternative by performing critical tasks in hazardous environments without endangering human personnel. These robots are typically equipped with flame sensors, gas detectors, and navigation systems that allow them to identify fire sources, avoid obstacles, and suppress flames independently. Their ability to operate in confined or unstable spaces makes them especially valuable in situations where human access is limited. Beyond improving safety, these systems also enhance operational efficiency. Robots can function continuously without fatigue, carry heavy equipment, and respond rapidly to developing fire threats. A notable example is the "Colossus" robot used during the 2019 Notre-Dame Cathedral fire, where it successfully operated in high-risk areas, assisting firefighters by suppressing flames and clearing debris (Peskoe-Yang, 2019).

Furthermore, the development of simulation tools such as Tinkercad allows engineers to design and test robotic systems virtually before physical implementation. These platforms are instrumental in refining control logic and sensor integration while reducing costs and development time.

In summary, autonomous robots offer a reliable and effective approach to modern fire suppression, combining safety, precision, and technological innovation (Wahyudi, 2022).

1.2. Context and literature

Fire remains one of the most persistent hazards in industrial, commercial, and residential environments. While conventional fire-fighting methods rely heavily on human intervention, they often expose personnel to high-risk situations and may delay rapid response. Autonomous fire-fighting robots offer a safer and more efficient alternative by detecting fire early and responding without direct human control. These robots integrate key components such as flame sensors, servo-

driven nozzles, and obstacle detection systems, enabling them to operate independently in complex or confined areas.

Simulation platforms like Autodesk Tinkercad have further accelerated the development of such systems by providing a safe, cost-effective environment for designing and testing embedded control logic. These tools are particularly valuable in education, where students can experiment with sensor-driven automation without needing physical hardware. Wahyudi et al. (Wahyudi, 2022) emphasize that real-time sensing and autonomous navigation significantly enhance robotic fire suppression capabilities, especially in hazardous or hard-to-access environments

1.3. Project objective

The aim of this project is to develop and simulate an autonomous fire-fighting robot capable of detecting and responding to small-scale fires without human intervention. The system is designed using the Arduino Uno microcontroller and implemented entirely within the Tinkercad simulation environment. Its core functions include detecting flames through LDR sensors, avoiding obstacles using ultrasonic sensors, and activating a simulated fire suppression system via a servo-controlled nozzle and LED indicator.

In addition, the project introduces a steering mechanism using a second servo motor, allowing the robot to change direction in a manner similar to real-world wheeled vehicles. This enhances the realism of its navigation behavior and supports more accurate obstacle avoidance. The broader objective is to demonstrate how readily available electronic components and open-source tools can be combined to create a cost-effective, fully autonomous robot for educational and prototyping purposes (Smith, 2023).

2. Material and methods

2.1. Hardware components

autonomous firefighting robot.

Saved

Component List

Download CSV

Name	Quantity	Component
UArduino Uno (Main Controller)	1	Arduino Uno R3
BAT9 volt dc battery	1	9V Battery
MLeft Rear Wheel (DC Motor) MRight Rear Wheel (DC Motor)	2	DC Motor
UL293D Motor Driver IC	1	H-bridge Motor Driver
RRight Flame Sensor (LDR) RCenter Flame Sensor (LDR) RLeft Flame Sensor (LDR)	3	Photoresistor
DISTRight Ultrasonic Sensor (HC-SR04) DISTLeft Ultrasonic Sensor (HC-SR04)	2	Ultrasonic Distance Sensor (4-pin)
R10kΩ Resistor - LDR Pull-down2 R10kΩ Resistor - LDR Pull-down1 R10kΩ Resistor - LDR Pull-down	3	10 kΩ Resistor
SERVONozzle Servo (Flame Targeting) SERVOSteering Servo (Front Wheels)	2	Positional Micro Servo
DPump LED (Simulated Water Spray)	1	Red LED
Rcurrent limiting resistor for led	1	220 Ω Resistor

Figure 1: showing list of components

The autonomous fire-fighting robot was assembled entirely in Autodesk Tinkercad (Autodesk, 2023) using off-the-shelf components chosen for their reliability and ease of integration with the Arduino Uno R3 (Arduino, 2024). Power is supplied by a simulated 9 V battery, representing a typical portable energy source for small mobile platforms.

Locomotion relies on two DC gear motors mounted as rear wheels. These are driven through an L293D motor driver IC, which permits bidirectional control and variable speed via pulse-width modulation—allowing the robot to advance, reverse, or pivot as needed.

Flame detection is achieved with three light-dependent resistors (LDRs) positioned at left, center, and right locations on the chassis. Each LDR forms a voltage divider with a 10 kΩ pull-down resistor, ensuring stable analog readings that reveal the direction of a light source (Kumar, 2024). Obstacle avoidance uses a pair of HC-SR04 ultrasonic sensors mounted at the front. By measuring echo return times, these units provide real-time distance data that the control logic uses to steer clear of impediments (MicroElectronika, 2023).

To mimic a car's steering mechanism, a micro-servo is fitted to the front assembly; it adjusts wheel angle for smooth turns. A second servo directs a nozzle toward detected flames. Finally, a red LED—protected by a 220 Ω resistor—serves as a mock pump indicator, illuminating when the robot "sprays" water.

This modular configuration combines sensing, actuation, and control into a low-cost prototype that can be tested and refined entirely in simulation, laying the groundwork for future physical implementation.

2.2. Circuit design

The circuit was developed and simulated using Autodesk Tinkercad (Autodesk, 2023), which allowed for accurate virtual prototyping without the need for physical components. The Arduino Uno serves as the central controller, interfacing with all sensors and actuators. LDRs are connected to analog inputs, while ultrasonic sensors are mapped to digital pins for obstacle detection. The L293D motor driver receives PWM signals to control DC motor speed and direction, and two servo motors handle steering and nozzle positioning. A red LED simulates the fire suppression system. A schematic view of the complete circuit, captured from Tinkercad, is attached below (Zhao, 2022).

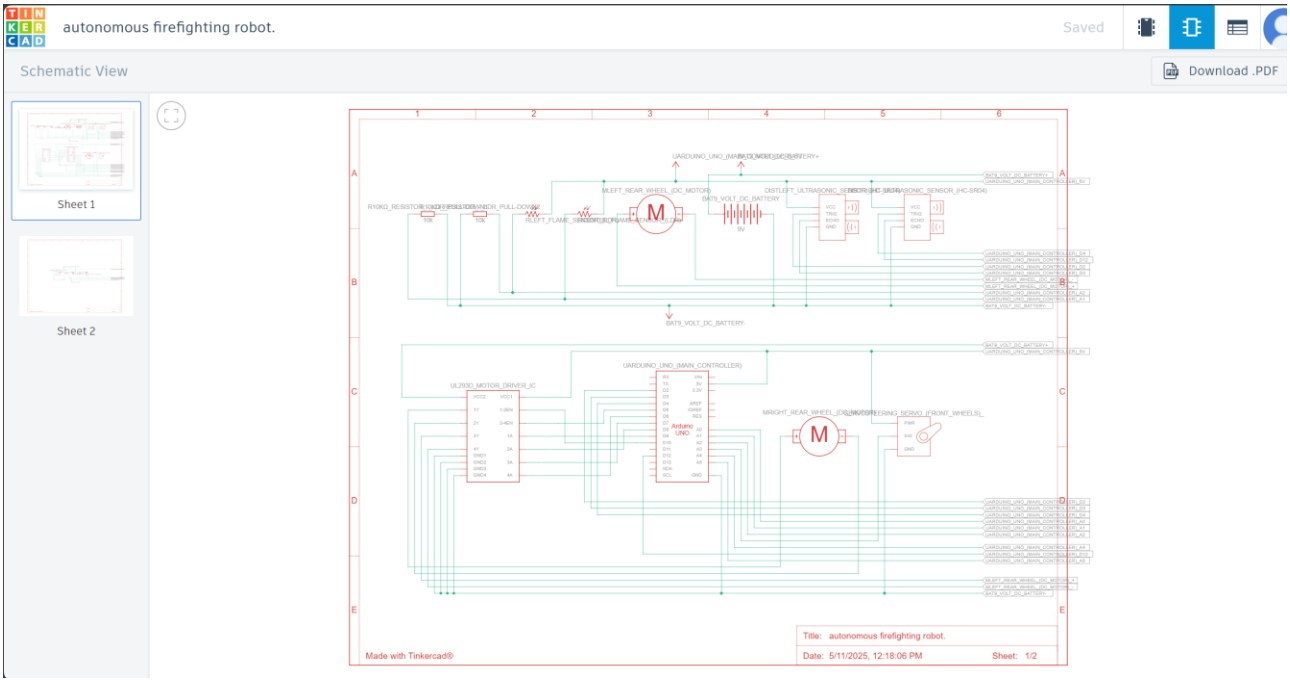

Figure 2: Schematic view of autonomous fire-fighting robot.

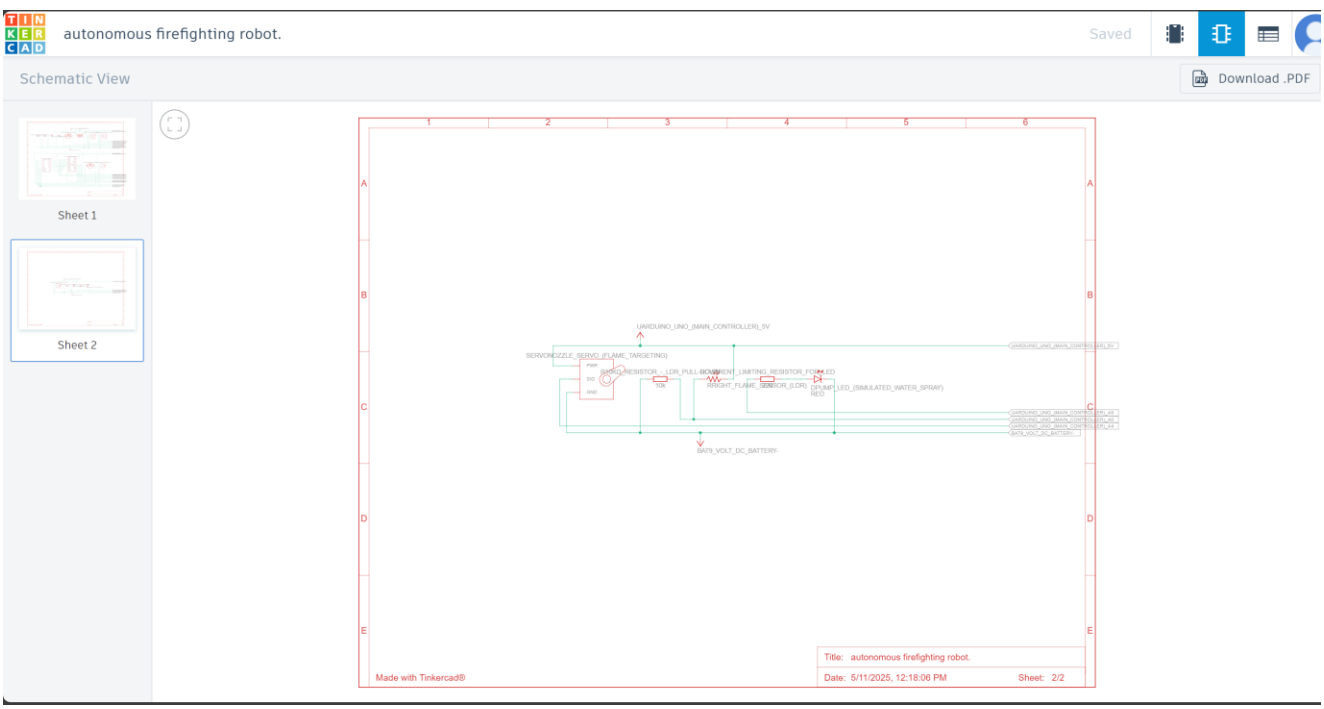

Figure 3: Schematic view 2

2.3. Control logic and software

The robot's control logic is programmed using Arduino C++ and deployed via the Arduino IDE (Monk, 2016). The software is responsible for managing two primary functions: flame detection and autonomous navigation.

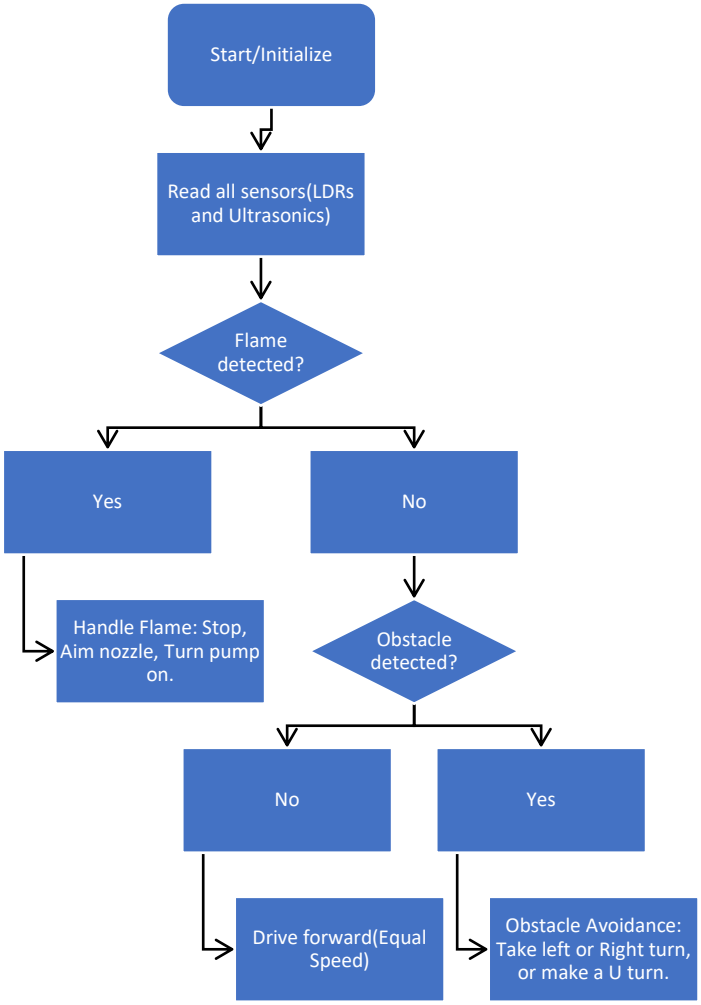

Figure 4: Flowchart showing the working process of the autonomous fire-fighting robot.

Flame sensing is achieved through the use of three LDR (Light Dependent Resistor) sensors, which are positioned to monitor light intensity on the left, center, and right sides of the robot. When a notable increase in brightness is detected by any of these sensors, the robot initiates a response sequence. It halts its movement, orients the nozzle—controlled by a servo motor—toward the direction of the detected flame, and activates an LED to simulate the discharge of water. The nozzle

is positioned at angles of 0°, 90°, or 180°, depending on whether the flame is located on the right, center, or left, respectively.

```
#include <Servo.h>

// Pins for the three LDR sensors
const int LDR_RIGHT_PIN   = A0;
const int LDR_CENTER_PIN = A1;
const int LDR_LEFT_PIN    = A2;

// Pump indicator (LED) and nozzle servo
const int PUMP_LED_PIN          = A5;
const int NOZZLE_SERVO_PIN      = A4;
Servo nozzleServo;

// Threshold above which we consider "flame detected"
const int FLAME_THRESHOLD = 600;

// Call this in setup()
void initFlameDetection() {
  pinMode(PUMP_LED_PIN, OUTPUT);
  digitalWrite(PUMP_LED_PIN, LOW);

  nozzleServo.attach(NOZZLE_SERVO_PIN);
  nozzleServo.write(90);   // Start with nozzle facing forward
}centre)

// This function is called in loop()
void handleFlameDetection() {
  // Read each LDR
  int rightValue  = analogRead(LDR_RIGHT_PIN);
  int centerValue = analogRead(LDR_CENTER_PIN);
  int leftValue   = analogRead(LDR_LEFT_PIN);

  // If the right sensor sees a bright flash...
  if (rightValue > FLAME_THRESHOLD) {
    // 1. Stop movement (assume other code handles motors)
    // 2. Centre steering
    // 3. Rotate nozzle to 0° (pointing right)
    // 4. Turn on pump LED for 5 seconds
    Serial.println("Flame detected on RIGHT");
    nozzleServo.write(0);
    digitalWrite(PUMP_LED_PIN, HIGH);
    delay(5000);
  }
  // If the left sensor sees a bright flash...
  else if (leftValue > FLAME_THRESHOLD) {
    Serial.println("Flame detected on LEFT");
    nozzleServo.write(180);
    digitalWrite(PUMP_LED_PIN, HIGH);
    delay(5000);
  }
  // If the centre sensor sees a bright flash...
  else if (centerValue > FLAME_THRESHOLD) {
    Serial.println("Flame detected at CENTRE");
    nozzleServo.write(90);
    digitalWrite(PUMP_LED_PIN, HIGH);
    delay(5000);
  }
  // No flame detected → turn off pump and reset nozzle
  else {
    digitalWrite(PUMP_LED_PIN, LOW);
    nozzleServo.write(90);
  }
}
```

Figure 5: Arduino code for initializes LDRs and checks for flames in real time.

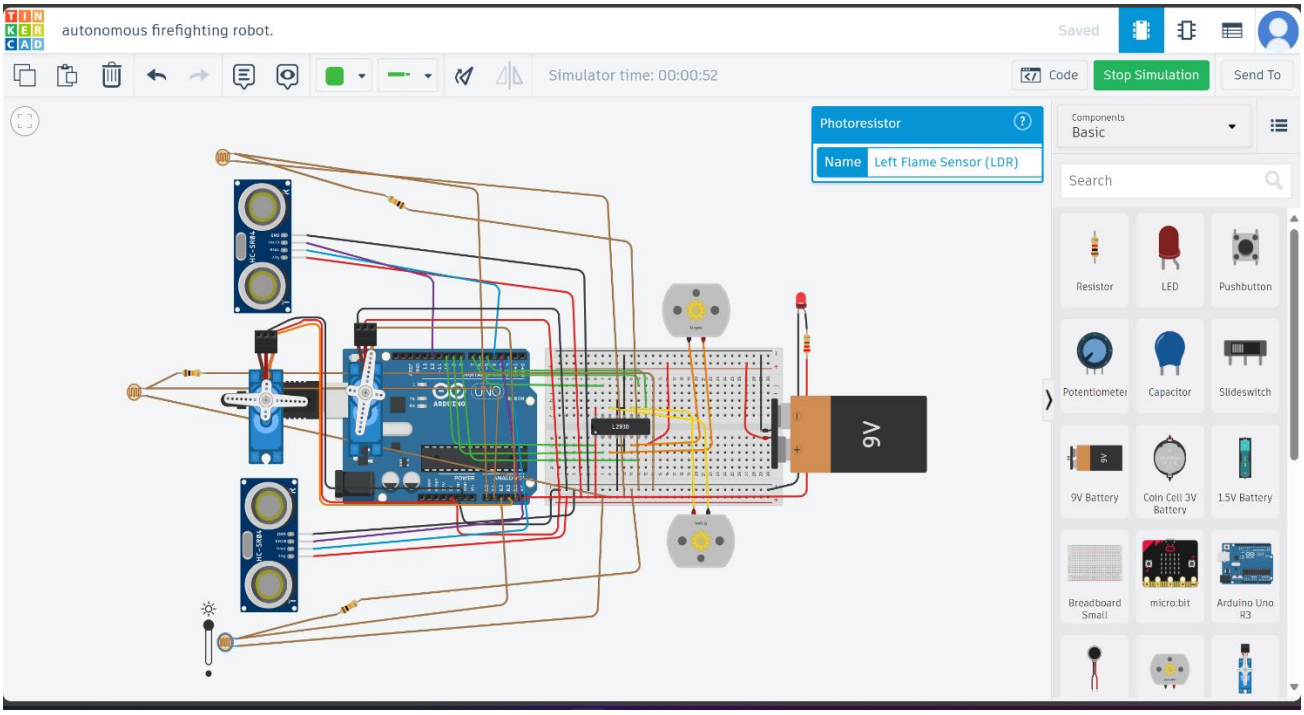

Figure 6: Simulation of flame sensors detecting fire and triggering the robot's firefighting response.

The robot's autonomous navigation relies on two ultrasonic sensors mounted on its left and right flanks to continuously gauge nearby obstacles. Whenever one sensor reports a distance below 10 cm, the steering servo deflects the front wheels away from the obstruction for a short interval before realigning them to the center. If both sensors register an obstacle within this threshold simultaneously, the robot first reverses and then executes a controlled U-turn by manipulating its drive motors. Under clear conditions, the vehicle proceeds forward with both motors running at the same speed, thereby maintaining a smooth and balanced trajectory (Autodesk, 2023).

```
#include <Servo.h>

// Pins for the two ultrasonic sensors
const int TRIG_LEFT_PIN  = 2;
const int ECHO_LEFT_PIN  = 3;
const int TRIG_RIGHT_PIN = 4;
const int ECHO_RIGHT_PIN = 12;

// Steering servo on the front wheels
const int STEERING_SERVO_PIN = A3;
Servo steeringServo;

// Motor driver pins (example)
const int ENA = 10, IN1 = 9, IN2 = 8;
const int ENB =  5, IN3 = 7, IN4 = 6;

// Parameters
const int OBSTACLE_DISTANCE_CM = 10;
const int TURN_TIME_MS         = 5000;
const int FORWARD_SPEED        = 150;
const int REVERSE_SPEED        = 150;

// Call this in setup()
void initAutonomous() {
  // Ultrasonic pins
  pinMode(TRIG_LEFT_PIN, OUTPUT); pinMode(ECHO_LEFT_PIN, INPUT);
  pinMode(TRIG_RIGHT_PIN, OUTPUT); pinMode(ECHO_RIGHT_PIN, INPUT);

  // Steering servo
  steeringServo.attach(STEERING_SERVO_PIN);
  steeringServo.write(90);  // Wheels straight ahead

  // Motor driver pins
  pinMode(ENA, OUTPUT); pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  pinMode(ENB, OUTPUT); pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);
}

// Reads distance (in cm) from an HC-SR04 ultrasonic module
long readDistance(int trigPin, int echoPin) {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  long duration = pulseIn(echoPin, HIGH, 30000);
  return duration * 0.034 / 2;
}

// Basic motor commands
void driveForward() {
  digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
  digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
  analogWrite(ENA, FORWARD_SPEED); analogWrite(ENB, FORWARD_SPEED);
}

void driveReverse() {
  digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
  digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);
  analogWrite(ENA, REVERSE_SPEED); analogWrite(ENB, REVERSE_SPEED);
}

void stopMotors() {
  digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW); digitalWrite(IN4, LOW);
  analogWrite(ENA, 0); analogWrite(ENB, 0);
}

// This function is called in loop()
void handleAutonomousMovement() {
  long distLeft  = readDistance(TRIG_LEFT_PIN, ECHO_LEFT_PIN);
  long distRight = readDistance(TRIG_RIGHT_PIN, ECHO_RIGHT_PIN);

  // Obstacle on right only → steer left briefly
  if (distRight < OBSTACLE_DISTANCE_CM && distLeft >= OBSTACLE_DISTANCE_CM) {
    stopMotors();
    Serial.println("Obstacle RIGHT → steering LEFT");
    steeringServo.write(135);
    delay(TURN_TIME_MS);
    steeringServo.write(90);
  }
  // Obstacle on left only → steer right briefly
  else if (distLeft < OBSTACLE_DISTANCE_CM && distRight >= OBSTACLE_DISTANCE_CM)
{   stopMotors();
    Serial.println("Obstacle LEFT → steering RIGHT");
    steeringServo.write(45);
    delay(TURN_TIME_MS);
    steeringServo.write(90);
  }
  // Obstacles on both sides → reverse and U-turn
  else if (distLeft < OBSTACLE_DISTANCE_CM && distRight < OBSTACLE_DISTANCE_CM) {
    Serial.println("Obstacles BOTH → reversing for U-turn");
    stopMotors();
    driveReverse();
    delay(1500);
    stopMotors();
    steeringServo.write(135);  // wheels turned left
    driveForward();
    delay(1200);
    steeringServo.write(90);   // realign straight
  }
  // No obstacles → drive straight ahead
  else {
    Serial.println("Path clear → driving forward");
    steeringServo.write(90);
    driveForward();
  }
}
```

Figure 7: Arduino code for obstacle detection using ultrasonic sensors.

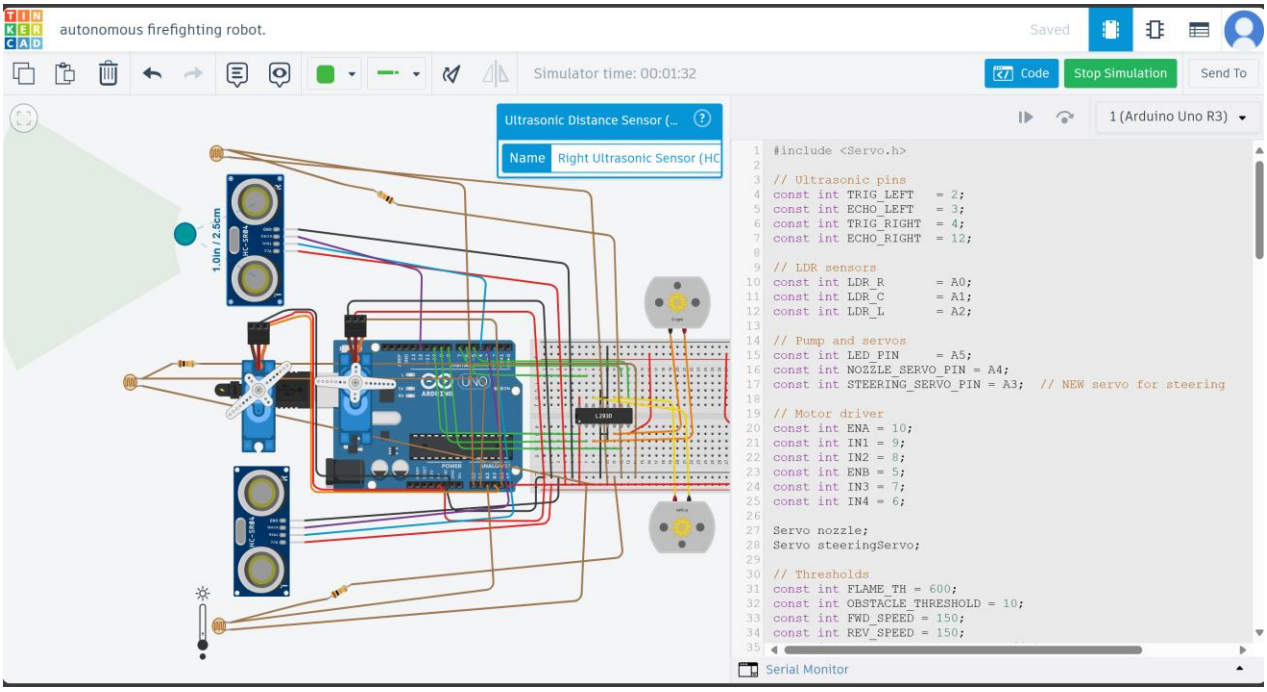

Figure 8: Simulation of ultrasonic sensors detecting obstacles and triggering the robot's automation response.

3. Results & Discussion

3.1. Simulation outcomes

The autonomous fire-fighting robot was designed within Autodesk Tinkercad, furnishing an interactive platform to evaluate sensor integration and control algorithms prior to physical assembly. In each simulation run, straight-line motion was maintained by delivering equal PWM signals to both drive motors. When one ultrasonic sensor detected an object closer than 10 cm, the steering servo deflected the front wheels away from the obstacle for a brief interval before recentering them. If both sensors simultaneously reported proximity below the threshold, the robot initiated a reverse maneuver, executed a controlled U-turn via differential motor actuation, and then resumed forward transit. Under unobstructed conditions, uniform motor speeds ensured stable progression. Subsequent flame-detection trials demonstrated that, upon an LDR reading exceeding

the predefined threshold, the robot halted, oriented its servo-driven nozzle to 0°, 90°, or 180° according to the flame's bearing, and activated an LED to simulate water discharge. Collectively, these results validate the efficacy of the autonomous navigation, obstacle-avoidance, and suppression routines. Link for simulation is attached below (Chen, 2022).

https://www.tinkercad.com/things/8jLVSEBx8tk-autonomous-firefighting-robot/editel?returnTo=https%3A%2F%2Fwww.tinkercad.com%2Fdashboard%2Fdesigns%2Fcircuits&sharecode=Fzvbetp2PXNSnp6aD_5OYnYgXA8cTrwvfL23fmr_7Nw

3.2. System strengths

The autonomous fire-fighting robot employs integrated flame and ultrasonic sensors to operate in real time within hazardous scenarios. Its modular framework—built around an Arduino Uno, an L293D motor driver, and SG90 servos—ensures straightforward experimentation, customization, and upkeep, thus supporting both educational objectives and economical prototyping. By harnessing the Autodesk Tinkercad simulation platform, we achieve full system validation without physical components, speeding up initial development while lowering risk. In essence, its autonomy, modularity, cost-effectiveness, and user-friendliness establish a foundation for instruction and inquiry.

3.3. Limitations and future work

Although the simulation yielded promising results, the system exhibits several real-world limitations. LDR sensors may offer unreliable flame detection when confronted with smoke or fluctuating ambient light, and the Tinkercad environment cannot emulate sensor noise, motor torque variability, or surface friction. Furthermore, the absence of wireless connectivity and advanced navigation reduces flexibility and precludes remote monitoring. To address these issues, future work should incorporate infrared flame detectors, add wireless control modules, implement intelligent pathfinding, camera should be attached to find path or update live situation and validate performance on actual hardware (Muratore, 2022).

3.4. Applications

Autonomous fire-fighting robots are well suited to environments where rapid response is critical and human access is restricted, such as industrial plants, warehouses, and power stations. By detecting and suppressing flames early, these systems minimize property damage and improve safety for emergency responders (Wahyudi, 2022). In academia, this Arduino-based prototype serves as a hands-on educational tool for teaching robotic sensing, control, and embedded programming. With enhancements like thermal cameras, wireless telemetry, and GPS integration, such robots could also support search-and-rescue missions and smart-city fire management, forming a core component of future resilient infrastructure (Mahmud, 2020).

4. Conclusion

This project has successfully demonstrated an Arduino-based autonomous fire-fighting robot, fully developed and tested within the Tinkercad simulation environment. By combining three LDR flame sensors, two ultrasonic rangefinders, an L293D motor driver, and dual servos for steering and nozzle control, the robot is capable of detecting fire sources, avoiding obstacles, and simulating targeted suppression. The modular hardware design and clear, priority-based control logic make the system both accessible and instructive, using only off-the-shelf components and open-source software. Simulation results confirmed that the robot reliably navigates its environment, responds promptly to hazards, and executes suppression routines as intended. While real-world implementation will require more robust sensors and refined navigation algorithms, this prototype lays the groundwork for further development. Future work might include integrating thermal cameras, wireless telemetry, and closed-loop motor control to enhance precision and adaptability in practical fire-response applications (Zhao, 2022).

References

Arduino, 2024. *Arduino Uno Rev3 — Tech Specs.* [Online] Available at: https://www.arduino.cc/en/Main/ArduinoBoardUno

Autodesk, 2023. *Tinkercad Circuits Overview.* [Online] Available at: https://www.tinkercad.com

Cacace, J. F. A. L. V. S. B. a. V. L., 2022. An autonomous ground robot to support firefighters' interventions in hazardous scenarios. *Journal of Field Robotics,* 39(2), p. 267–289.

Chen, Y. a. Z. L., 2022. *Simulation of fire fighter swarm robotics in MATLAB and robot e.* s.l., s.n.

Kumar, A. a. S. R., 2024. *Arduino UNO-based robotic system for combating fires.* s.l., s.n.

Mahmud, M. G. C. a. F. B., 2020. Fire detection and fighting robot: A review. *Procedia Computer Science,* Volume 170, p. 491–498.

MicroElectronika, 2023. *HC-SR04 Datasheet – Ultrasonic Distance Sensor.* [Online] Available at: https://www.microelectronika.com/product/hc-sr04-ultrasonic-sensor

Monk, S., 2016. *Programming Arduino: Getting Started with Sketches.* 2nd Edition ed. s.l.:McGraw-Hill Education.

Muratore, F. R. F. T. G. Y. W. a. P. J., 2022. Robot Learning From Randomized Simulations: A Review. *Frontiers in Robotics and AI,* 9(https://www.frontiersin.org/articles/10.3389/frobt.2022.799893/full), p. Article 799893.

Peskoe-Yang, L., 2019. *Colossus – The Firefighting Robot That Helped Save Notre-Dame.* [Online]
Available at: https://spectrum.ieee.org/colossus-the-firefighting-robot-that-helped-save-notre-dame

Smith, J. a. L. K., 2023. Autonomous firefighting using a quadruped robot. *Transactions of the Canadian Society for Mechanical Engineering,* 47(3), p. 345–356.

Wahyudi, A. A. M. a. B. H., 2022. A survey on autonomous fire-fighting robots. *International Journal of Advanced Robotic Systems,* 19(1), p. 1–10.

Wahyudi, W. P. I. a. S. R., 2022. Design of autonomous fire-fighting robot using flame sensor and ultrasonic sensor with Arduino. *Journal of Robotics and Control (JRC),* 3(5), p. 563–570.

Zhao, H. a. W. Y., 2022. Design of intelligent fire-fighting robot based on multi-sensor fusion. *Robotics and Autonomous Systems,* Volume 146, p. 103–112.

Zhao, H. a. W. Y., 2022. Design of intelligent fire-fighting robot based on multi-sensor fusion. *Robotics and Autonomous Systems,* Volume 146, p. 103–112.